Міністерство освіти і науки України

Національний університет «Одеська політехніка»

Інститут комп’ютерних систем

Кафедра інформаційних систем

Лабораторна робота №2

З дисципліни: «Програмування мобільних пристроїв на базі ОС Android»

Тема: «Списки та управління станом у Jetpack Compose»

Виконала: студент групи АІ-231

Акімов А.І.

Перевірили: Годовиченко М.А.

Одеса 2026

Мета роботи: Розробити одноекранний Android-застосунок «Список покупок» для запису і відстеження з використанням Jetpack Compose, ViewModel та StateFlow.

Завдання:

Варіант 1. Список покупок

Завдання. Розробити одноекранний Android-додаток «Список покупок» з використанням Jetpack Compose та архітектурного компонента 'ViewModel'.

Функціональні вимоги:

- Поле введення (TextField) для назви нового товару;
- Кнопка «Додати» — додає товар до списку; неактивна якщо поле порожнє;
- LazyColumn — відображає список товарів у вигляді карток;
- Тап на картку товару — перемикає стан «куплено / не куплено» (перекреслений текст);
- Лічильник «Залишилось купити: N» у заголовку — оновлюється реактивно.

Приклад структури файлів:

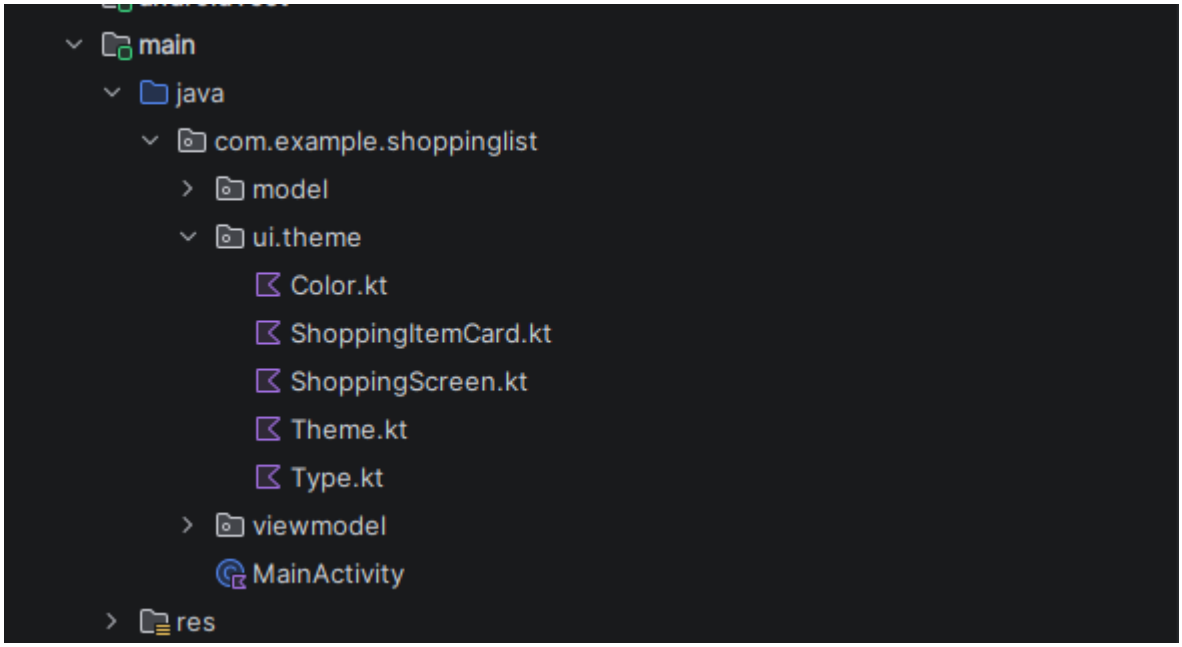
- ui/ShoppingScreen.kt — головний екран (Composable)
- ui/ShoppingItem.kt — окрема Composable-картка товару
- viewmodel/ShoppingViewModel.kt — логіка і стан
- model/ShoppingItem.kt — data class

Технічні вимоги:

- Стан списку — MutableStateFlow<List> у ViewModel;
- UI підписується через collectAsStateWithLifecycle();
- Лічильник — похідний StateFlow: items.map { list -> list.count { !it.isBought } };
- Заборонено зберігати стан безпосередньо у Composable (тільки ViewModel);
- Кожен товар має унікальний id (UUID або автоінкремент) — обов'язково для key у LazyColumn.

ХІД РОБОТИ

Проєкт створено в Android Studio з використанням шаблону Empty Activity (Jetpack Compose). Структура файлів:



model/ShoppingItem.kt — data class моделі товару.

viewmodel/ShoppingViewModel.kt — ViewModel зі StateFlow.

ui/ShoppingScreen.kt — головний екран.

ui/ShoppingItemCard.kt — картка товару.

Модель даних — ShoppingItem.kt

Створено data class для зберігання інформації про товар у списку покупок:

```
data class ShoppingItem(
    val id: String = UUID.randomUUID().toString(),
    val name: String,
    val quantity: Int = 1,
    val isBought: Boolean = false
)
```

ShoppingViewModel

Реалізовано ViewModel з використанням MutableStateFlow для зберігання списку товарів. Кількість некуплених товарів обчислюється як похідний Flow:

```
val remainingCount: StateFlow<Int> = _items
    .map { list -> list.count { !it.isBought } }
    .stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5_000),
        initialValue = 0
    )
```

Стан кожного товару (куплено/не куплено) керується через функцію toggleItem, яка приймає id товару та інвертує значення поля isBought.

ShoppingItemCard.kt
Створено окремий Composable-компонент для відображення картки товару. Картка містить круглий індикатор стану, назву товару та кількість. Підтримує анімацію зміни кольору через animateColorAsState та обробку кліку через параметр onToggle.

ShoppingScreen.kt
Реалізовано головний екран застосунку, який включає:

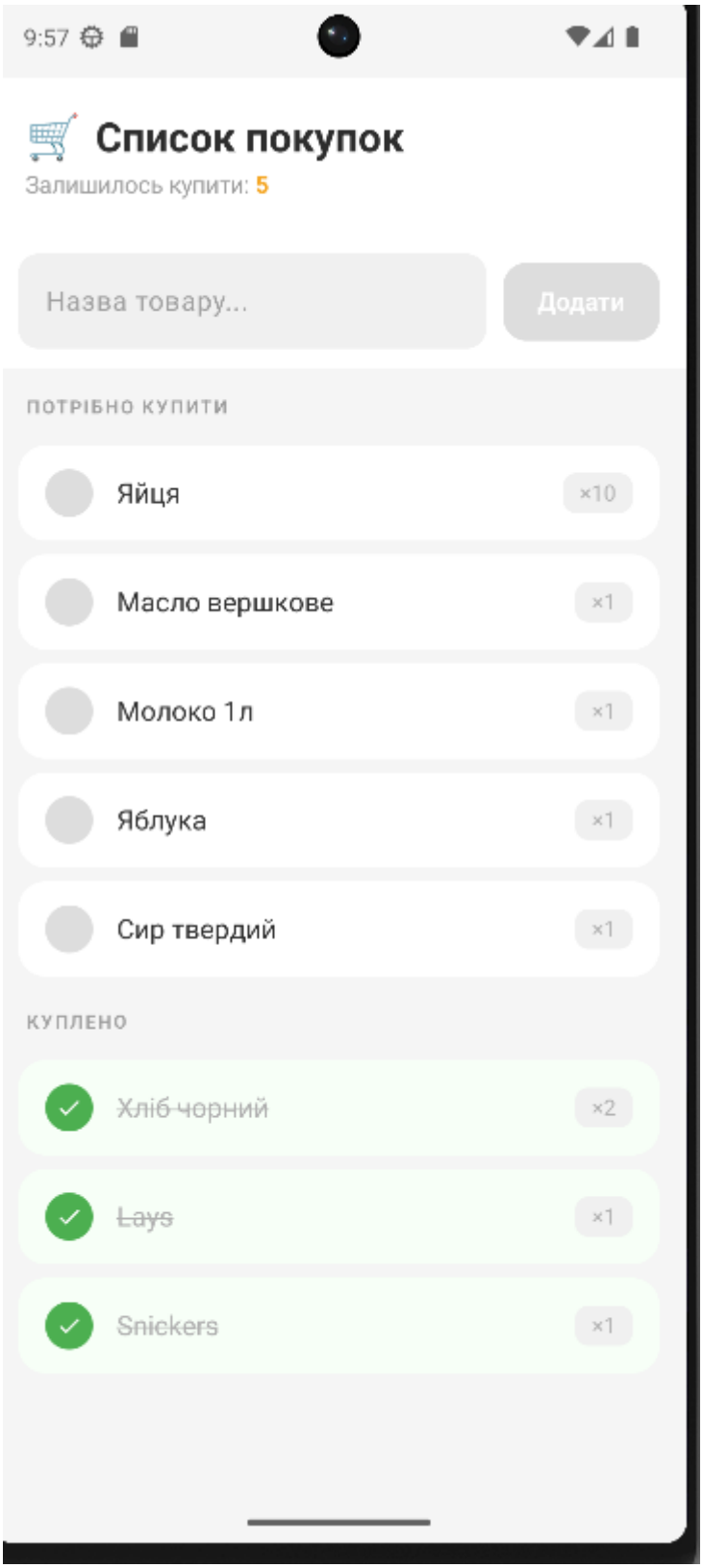
- Заголовок з реактивним лічильником некуплених товарів (collectAsStateWithLifecycle)
- Форму введення з полем TextField та кнопкою «Додати»
- Кнопку «Додати» з валідацією — активна лише якщо поле не порожнє
- LazyColumn зі списком товарів, розділеним на секції «ПОТРІБНО КУПИТИ» та «КУПЛЕНО»
- Анімацію появи елементів через AnimatedVisibility

StateFlow та реактивний лічильник
Список товарів зберігається як MutableStateFlow<List<ShoppingItem>>. Кількість некуплених товарів є похідним StateFlow, що автоматично перераховується при будь-якій зміні списку. У Composable підписка відбувається через collectAsStateWithLifecycle().

Валідація форми
Кнопка «Додати» активується лише якщо поле введення не порожнє:

```
enabled = inputText.isNotBlank()
```

Запуск та тестування
Застосунок успішно запущено на емуляторі. Перевірено всі функціональні вимоги: додавання товарів, реактивне оновлення лічильника, позначення товарів як куплених з анімацією, розділення списку на секції.


9:57
Список покупок
Залишилось купити: 5
Назва товару...
Додати
ПОТРІБНО КУПИТИ
Яйця
×10
Масло вершкове
×1
Молоко 1л
×1
Яблука
×1
Сир твердий
×1
КУПЛЕНО
Хліб чорний
×2
Lays
×1
Snickers
×1

Bounty

Додати

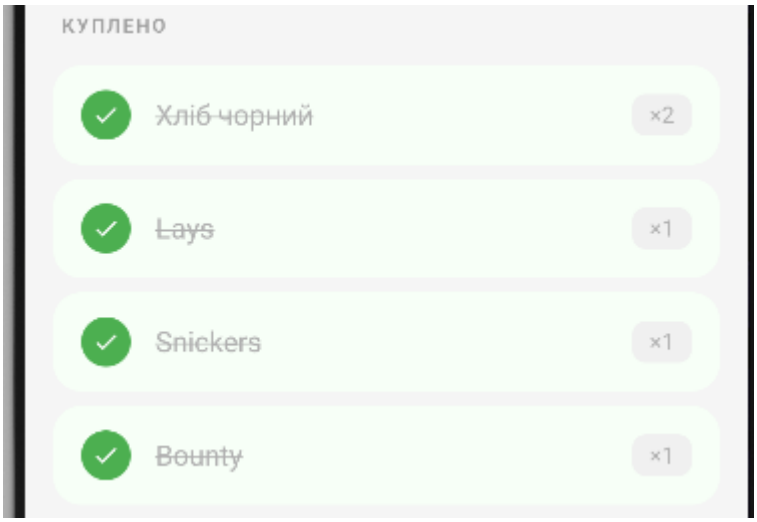
ПОТРІБНО КУПИТИ

- Яйця ×10
- Масло вершкове ×1
- Молоко 1л ×1
- Яблука ×1
- Сир твердий ×1

ПОТРІБНО КУПИТИ

- Яйця ×10
- Масло вершкове ×1
- Молоко 1л ×1
- Яблука ×1
- Сир твердий ×1
- Bounty ×1

КУПЛЕНО



ВИСНОВОК

У ході виконання лабораторної роботи було розроблено одноекранний Android-застосунок «Список покупок» з використанням Jetpack Compose та архітектурного компонента ViewModel. У процесі роботи набуто практичних навичок реактивного управління станом через StateFlow, побудови динамічних списків за допомогою LazyColumn з розділенням на секції, а також реалізації анімацій через AnimatedVisibility та animateColorAsState. Реалізовано валідацію форми введення, позначення товарів як куплених з візуальним відображенням через закреслення тексту та зміну кольору картки, а також реактивне оновлення лічильника некуплених товарів у заголовку. Уся логіка стану винесена у ViewModel відповідно до архітектурних вимог завдання. Застосунок успішно протестовано на емуляторі.

ДОДАТОК

Посилання на репозиторій:

https://github.com/sawanda19/ShoppingList_LR2